

Sección 3: Administrar la memoria, el estado y la ejecución (10-15%)

🔑 **Esta sección evalúa tu capacidad para gestionar cómo un agente de IA recuerda información, mantiene su progreso y evita desviarse de sus objetivos durante ejecuciones prolongadas.** Cubre los tres tipos de memoria (corto plazo, largo plazo, externa), las estrategias de persistencia del estado, la detección de deriva de contexto y la continuidad entre herramientas y entornos. Aunque representa un porcentaje menor del examen (10-15%), los conceptos aquí tratados son transversales y se conectan con las demás secciones.

3.1 Implementar estrategias de memoria del agente

Conceptos clave

Los tres tipos de memoria de un agente

La memoria de un agente de IA no es un concepto monolítico. Se divide en tres categorías fundamentales, cada una con un propósito, alcance y ciclo de vida diferente:

Tipo de memoria	Definición	Alcance temporal	Ejemplo en GitHub Copilot
Memoria a corto plazo	Contexto de la conversación o sesión actual	Dura solo mientras la sesión esté activa	La ventana de chat de Copilot, el historial de mensajes en una sesión del agente de codificación
Memoria a largo plazo	Información persistente que sobrevive entre sesiones	Persiste durante semanas, meses o indefinidamente	Copilot Memory (hechos de repositorio y preferencias de usuario), archivos de instrucciones personalizadas
Memoria externa	Fuentes de datos fuera del modelo que el agente consulta activamente	Depende de la fuente; puede ser estática o dinámica	Archivos del repositorio, bases de datos, APIs, resultados de búsqueda de código, documentación

💡 **Modelo mental:** La memoria a corto plazo es lo que el agente "está pensando ahora", la memoria a largo plazo es lo que "ya sabe desde antes" y la memoria externa es lo que "puede ir a buscar cuando lo necesite".

Memoria a corto plazo en detalle

La memoria a corto plazo es el **contexto de la conversación actual**. En GitHub Copilot, esto incluye:

- Los mensajes anteriores en el chat (prompts del usuario y respuestas del agente).
- Los archivos abiertos o referenciados durante la sesión.
- Los resultados de herramientas invocadas (lectura de archivos, ejecución de comandos, búsquedas).

- Las decisiones tomadas y los pasos completados dentro de la sesión.

Características:

- **Volátil:** Se pierde cuando la sesión termina.
- **Limitada:** Tiene un tamaño máximo (ventana de contexto del modelo). Cuando se excede, los mensajes más antiguos se descartan o resumen.
- **Inmediata:** Es la memoria más rápida de acceder; ya está en el contexto del modelo.

Riesgo principal: Si la sesión es muy larga, la memoria a corto plazo puede desbordarse. Los primeros pasos y decisiones se pierden, lo que causa **deriva de contexto** (ver sección 3.2).

Memoria a largo plazo en detalle

La memoria a largo plazo permite que el agente retenga información **entre sesiones**. En el ecosistema de GitHub Copilot, existen dos mecanismos principales:

1. Copilot Memory (Memorias de Copilot)

Copilot Memory almacena **hechos** que el agente utiliza para personalizar sus respuestas. Se divide en dos niveles:

Nivel	Contenido	Quién lo ve	Disponibilidad
Repositorio	Convenciones de código, decisiones arquitectónicas, comandos de compilación, reglas del proyecto	Todos los usuarios con Copilot Memory habilitado en ese repositorio	GitHub Copilot Business/Enterprise (administrado por la organización)
Usuario	Preferencias personales de estilo de codificación, patrones de flujo de trabajo	Solo el usuario individual	Copilot Pro/Pro+ únicamente

Características de Copilot Memory:

- **Almacena hechos con citaciones al código fuente.** Cada memoria hace referencia al archivo o contexto de donde se extrajo.
- **Validación contra la rama actual:** Antes de usar una memoria, Copilot verifica que siga siendo relevante comparándola con el estado actual del código.
- **Auto-eliminación tras 28 días sin uso:** Si una memoria no se utiliza durante 28 días, se elimina automáticamente. Esto previene la acumulación de información obsoleta.
- **Consumidores:** Copilot Memory es utilizado por el agente de codificación en la nube (Copilot coding agent), Copilot Code Review y Copilot CLI.

2. Archivos de instrucciones personalizadas (Custom Instructions)

Son archivos estáticos que proporcionan contexto persistente al agente:

Archivo	Alcance	Ubicación
---------	---------	-----------

Archivo	Alcance	Ubicación
<code>.github/copilot-instructions.md</code>	Todo el repositorio	Raíz del directorio <code>.github/</code>
<code>*.instructions.md</code> (archivos <code>.instructions.md</code>)	Directorio o conjunto de archivos específico	Cualquier directorio del proyecto

Estos archivos actúan como "memoria declarativa" del agente: le dicen qué convenciones seguir, qué patrones utilizar, qué evitar, etc. A diferencia de Copilot Memory, no se auto-eliminan y requieren mantenimiento manual.

Memoria externa en detalle

La memoria externa es toda información que el agente **no tiene cargada en su contexto pero puede obtener activamente**:

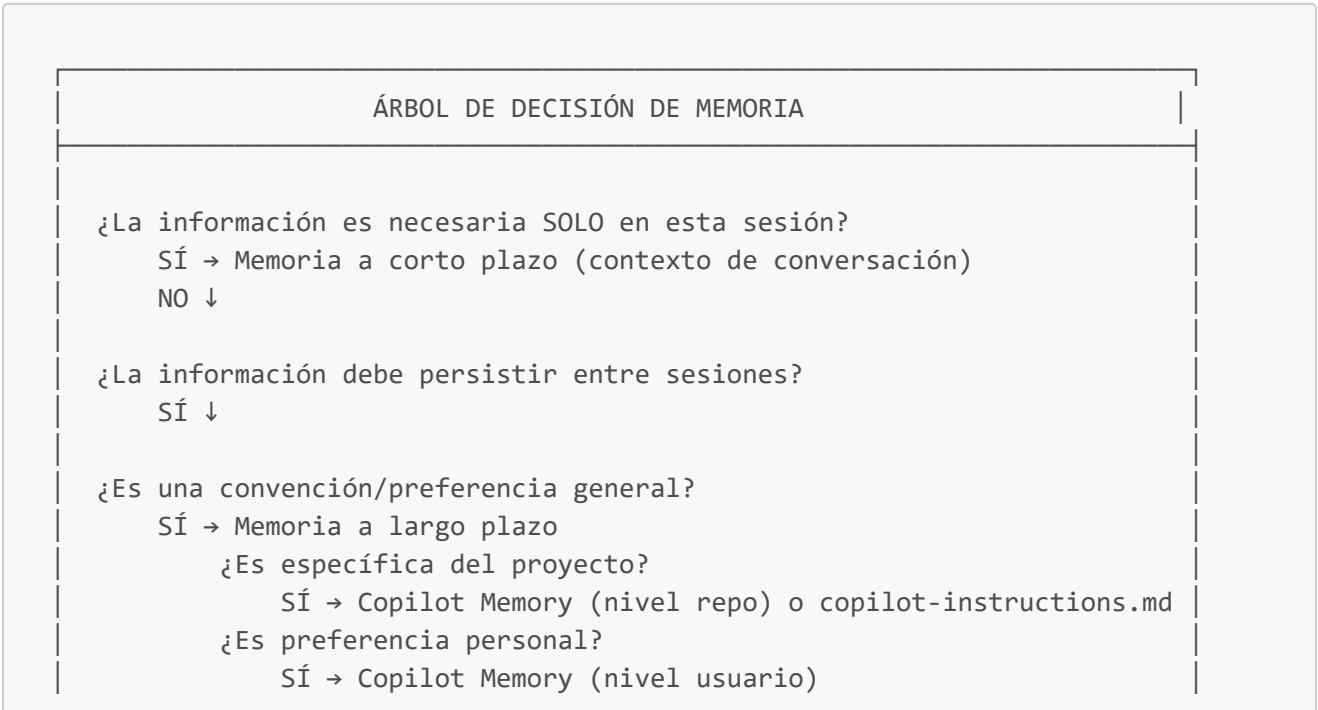
- **Archivos del repositorio:** El agente lee código fuente, configuraciones, documentación.
- **Resultados de búsqueda de código:** Consultas sobre símbolos, funciones, patrones en el proyecto.
- **APIs y servicios:** Consultas a servicios externos mediante herramientas MCP u otras integraciones.
- **Bases de datos:** Almacenamiento estructurado que el agente puede leer y escribir.
- **Issues, PRs y discusiones de GitHub:** Historial de decisiones y contexto del proyecto.

Ventaja clave: La memoria externa no está limitada por la ventana de contexto del modelo. El agente puede acceder a cantidades arbitrarias de información, siempre que tenga las herramientas necesarias.

Riesgo: El acceso a memoria externa tiene un costo en tiempo (latencia) y puede introducir información contradictoria si las fuentes no están sincronizadas.

Elegir entre los tres tipos de memoria

La elección del tipo de memoria depende de varios factores:



NO ↓

¿Es un dato estructurado, un archivo o un recurso consultable?

SÍ → Memoria externa (archivos, BD, APIs)

Escenarios prácticos de selección

Escenario	Tipo de memoria recomendado	Justificación
El agente necesita recordar que el proyecto usa tabs en vez de espacios	Largo plazo (Copilot Memory repo o <code>copilot-instructions.md</code>)	Es una convención persistente del proyecto
El agente está resolviendo un bug y ha leído 5 archivos relevantes	Corto plazo (contexto de sesión)	Solo se necesita durante la resolución del bug
El agente necesita consultar el esquema de la base de datos	Externa (leer archivos de migración o consultar la BD)	Es información estructurada que cambia con el tiempo
Un desarrollador prefiere documentación en formato JSDoc	Largo plazo (Copilot Memory usuario)	Es una preferencia personal entre sesiones
El agente debe verificar si un endpoint ya existe	Externa (buscar en el código fuente)	Requiere consulta activa del estado actual del repositorio

Limitar la memoria del agente a información relevante

Un error común es sobrecargar al agente con información irrelevante. La **memoria efectiva** es aquella que se limita a lo que el agente necesita para la tarea actual.

Estrategias de filtrado de memoria

1. Principio de mínimo contexto necesario

Solo proporcionar al agente la información directamente relevante para su tarea actual. Cada pieza de contexto innecesario consume espacio en la ventana de contexto y puede confundir al modelo.

2. Segmentación por archivos `.instructions.md`

Usar archivos de instrucciones a nivel de directorio para que el agente solo reciba las instrucciones relevantes cuando trabaja en esa parte del proyecto:

```
proyecto/
├── .github/
│   └── copilot-instructions.md    ← Reglas generales del proyecto
├── frontend/
│   └── .instructions.md          ← Reglas específicas de frontend (React,
```

```
CSS, etc.)
├─ backend/
│   └─ .instructions.md      ← Reglas específicas de backend (API, BD,
etc.)
└─ infra/
    └─ .instructions.md      ← Reglas de infraestructura (Terraform,
    Docker, etc.)
```

3. Gestión activa de Copilot Memory

- Revisar periódicamente las memorias almacenadas para eliminar las que ya no son relevantes.
- Aprovechar la auto-eliminación a los 28 días: las memorias que no se usan se purgan solas.
- Evitar memorias genéricas que aplican a todos los proyectos (son ruido).

4. Resumen progresivo en sesiones largas

En lugar de mantener todo el historial de la conversación, resumir periódicamente los pasos completados y las decisiones tomadas, descartando los detalles intermedios.

Definir reglas de expiración, eliminación y restablecimiento de memoria

Expiración automática

Mecanismo	Regla de expiración	Controlado por
Copilot Memory	28 días sin uso → eliminación automática	GitHub (automático, no configurable por el usuario)
Contexto de sesión	Fin de la sesión o desbordamiento de la ventana de contexto	El modelo/plataforma
Archivos de instrucciones	No expiran; persisten mientras exista el archivo	El equipo de desarrollo (mantenimiento manual)
Memoria externa (archivos)	No expiran; persisten mientras exista la fuente	Depende de la fuente (git, BD, API)

Eliminación manual

- **Copilot Memory:** Los usuarios pueden eliminar memorias individuales desde la configuración de Copilot. Los administradores de organización pueden gestionar las memorias a nivel de repositorio.
- **Archivos de instrucciones:** Se eliminan borrando o modificando el archivo correspondiente en el repositorio.
- **Contexto de sesión:** Se reinicia creando una nueva sesión de chat.

Restablecimiento de memoria

El restablecimiento es necesario cuando la memoria del agente contiene información incorrecta o desactualizada:

1. **Restablecimiento suave:** Corregir las memorias específicas que son erróneas, manteniendo las válidas.
2. **Restablecimiento completo:** Eliminar todas las memorias y comenzar desde cero (útil cuando hay corrupción generalizada del contexto).
3. **Restablecimiento por validación:** Copilot Memory realiza esto automáticamente al validar cada memoria contra la rama actual antes de usarla. Si el código al que hace referencia una memoria ya no existe, esa memoria no se aplica.

Aplicación práctica en GitHub

Ejemplo: Configurar memoria para un proyecto TypeScript

```
<!-- .github/copilot-instructions.md -->
# Instrucciones del proyecto

## Stack tecnológico
- TypeScript 5.x con strict mode
- React 18 con hooks (no class components)
- PostgreSQL con Prisma ORM

## Convenciones de código
- Usar camelCase para variables y funciones
- Usar PascalCase para tipos, interfaces y componentes
- Archivos de componentes: PascalCase.tsx
- Tests junto al archivo fuente: archivo.test.ts

## Comandos de desarrollo
- Build: `npm run build`
- Tests: `npm test`
- Lint: `npm run lint`
```

```
<!-- backend/.instructions.md -->
# Instrucciones del backend

- Todas las rutas API siguen el patrón REST
- Usar middleware de autenticación en rutas protegidas
- Validar inputs con Zod schemas
- Manejar errores con el patrón Result<T, Error>
```

Ejemplo: Gestión de Copilot Memory

En una sesión de Copilot Chat, el agente podría crear memorias como:

Memoria	Tipo	Citación
"Este proyecto usa ESLint con la config de Airbnb"	Repositorio	<code>.eslintrc.json</code>

Memoria	Tipo	Citación
"Las migraciones de BD se ejecutan con <code>npx prisma migrate dev</code> "	Repositorio	<code>package.json</code>
"Prefiero que los mensajes de commit sigan Conventional Commits"	Usuario	(preferencia personal)

Estas memorias se revalidan automáticamente:

- Si `.eslintrc.json` cambia a la config estándar, la memoria sobre Airbnb se invalida.
- Si el usuario no usa la memoria sobre Conventional Commits en 28 días, se elimina.

Puntos clave para el examen

- ☒ **Tres tipos de memoria:** corto plazo (sesión), largo plazo (Copilot Memory + instrucciones), externa (archivos, BD, APIs).
- ☒ **Copilot Memory tiene dos niveles:** repositorio (visible para todos) y usuario (solo visible para el individuo, requiere Copilot Pro/Pro+).
- ☒ **Auto-eliminación a los 28 días sin uso:** es un mecanismo automático de Copilot Memory, no configurable.
- ☒ **Validación contra la rama actual:** Copilot Memory verifica que los hechos sigan siendo válidos antes de aplicarlos.
- ☒ **Archivos `.instructions.md` no expiran:** requieren mantenimiento manual por el equipo.
- ☒ **El principio de mínimo contexto:** limitar la memoria al contenido relevante para la tarea reduce ruido y mejora precisión.
- ☒ **Consumidores de Copilot Memory:** agente de codificación en la nube, Copilot Code Review y Copilot CLI.
- ☒ **Memoria segmentada:** usar archivos de instrucciones a nivel de directorio para dar contexto específico.

3.2 Persistir el estado del agente y administrar la deriva de contexto

Conceptos clave

¿Qué es el "estado" de un agente?

El estado del agente es el conjunto completo de información que define **dónde se encuentra en la ejecución de una tarea**. Incluye:

- **Progreso de la tarea:** ¿Qué pasos se han completado? ¿Qué falta?
- **Decisiones tomadas:** ¿Qué alternativas se evaluaron? ¿Por qué se eligió una?
- **Artefactos generados:** Archivos creados, modificaciones realizadas, tests escritos.
- **Contexto acumulado:** Información descubierta durante la ejecución (bugs encontrados, dependencias identificadas, etc.).

¿Por qué es necesario persistir el estado?

Los agentes pueden interrumpirse por múltiples razones:

- **Timeouts:** Ejecuciones que exceden el límite de tiempo.
- **Errores irre recuperables:** Fallos que terminan la sesión.
- **Intervención humana:** El usuario pausa o cancela la tarea.
- **Límites de contexto:** La ventana de contexto se llena y se pierde información.
- **Despliegues o reinicios:** La infraestructura del agente se reinicia.

Sin persistencia del estado, el agente tendría que **empezar desde cero** cada vez. Esto es ineficiente y puede llevar a resultados inconsistentes.

Estrategias de persistencia del estado en GitHub

GitHub proporciona varios mecanismos nativos para persistir el estado de un agente:

1. Git Commits (el mecanismo principal)

Cada commit es un punto de persistencia del estado. El agente de codificación de Copilot hace commits frecuentes para:

- Guardar el progreso de manera incremental.
- Crear puntos de restauración en caso de error.
- Documentar qué se hizo y por qué (en el mensaje del commit).

```
feat: add user authentication middleware

- Implemented JWT validation middleware
- Added role-based access control
- Connected to existing user service

Next steps: Add rate limiting and session management
```

2. Descripciones de Pull Requests

La descripción del PR actúa como un **resumen del estado acumulado** del agente:

- Qué problema se está resolviendo.
- Qué enfoque se eligió y por qué.
- Qué se ha completado y qué falta.
- Decisiones de diseño y trade-offs.

3. Comentarios en Issues

Los comentarios en issues permiten:

- Documentar el progreso de una tarea a lo largo del tiempo.
- Registrar decisiones intermedias con su justificación.
- Crear un historial auditable de la ejecución del agente.

4. Artefactos de sesión del agente

El agente de codificación de Copilot genera un plan interno (plan.md o equivalente) que:

- Desglosa la tarea en pasos.
- Marca los pasos completados.
- Registra los hallazgos intermedios.

Capturar el progreso como artefactos duraderos

Los **artefactos duraderos** son piezas de información que sobreviven más allá de la sesión actual del agente. Los más importantes son:

Artefacto	Durabilidad	Visibilidad	Ejemplo
Commit de git	Permanente (en el historial del repo)	Todos los colaboradores	<code>fix: resolve null pointer in auth flow</code>
Descripción del PR	Dura mientras el PR exista	Todos los colaboradores	Plan detallado del cambio
Comentario en issue	Permanente	Todos los colaboradores	"He investigado X, Y, Z. La causa raíz es..."
Rama de trabajo	Hasta que se elimine	Todos los colaboradores	<code>copilot/fix-123</code>
Archivo de plan	Dura mientras la sesión esté activa	Solo el agente	Lista de pasos con estado
Copilot Memory	28 días sin uso	Según el nivel (repo/usuario)	"El proyecto usa PostgreSQL 15"

Práctica recomendada: El agente debe hacer commits frecuentes con mensajes descriptivos. Cada commit debe ser atómico (un cambio lógico completo) y autocontenido.

¿Qué es la deriva de contexto?

La **deriva de contexto** (context drift) es el fenómeno por el cual un agente **pierde gradualmente la alineación con su tarea original** durante ejecuciones prolongadas.

Causas de la deriva de contexto

CAUSAS DE LA DERIVA DE CONTEXTO

1. DESBORDAMIENTO DE LA VENTANA DE CONTEXTO

El historial de la conversación excede el límite del modelo.
Los mensajes más antiguos (con el objetivo original) se descartan.
El agente "olvida" qué estaba haciendo originalmente.

2. ACUMULACIÓN DE CONTEXTO IRRELEVANTE

El agente investiga múltiples caminos, acumula información.
La señal (objetivo original) se diluye en el ruido.
Las decisiones nuevas contradicen las anteriores.

3. CAMBIOS EN EL ENTORNO

Otros desarrolladores hacen cambios en la rama base.
Las dependencias se actualizan.
Los tests o CI/CD cambian sus requisitos.

4. SESCO DE RECENCIA (RECENCY BIAS)

El agente prioriza la información más reciente en su contexto.
Si la información reciente es un error o tangente, el agente puede alejarse del objetivo original.

Síntomas de la deriva de contexto

- El agente comienza a resolver un problema diferente al solicitado.
- Las soluciones generadas contradicen decisiones tomadas anteriormente en la misma sesión.
- El agente repite pasos que ya completó.
- El agente introduce cambios no solicitados en áreas no relacionadas.
- La calidad de las respuestas se degrada progresivamente.

Detectar y corregir la deriva

Estrategias de detección

1. Anclas de objetivo (Goal Anchoring)

Definir claramente el objetivo al inicio de la sesión y referenciarlo periódicamente. En instrucciones personalizadas, se puede incluir:

```
<!-- En las instrucciones del agente -->  
Antes de cada acción significativa, verifica que esté alineada  
con el objetivo original de la tarea.
```

2. Checkpoints de validación

Insertar puntos de verificación en la ejecución donde el agente debe confirmar:

- ¿Estoy todavía trabajando en la tarea original?
- ¿Las decisiones recientes son consistentes con las anteriores?
- ¿He introducido cambios fuera del alcance de la tarea?

3. Revisión humana periódica (Human-in-the-loop)

En tareas largas, solicitar revisión humana en puntos clave:

- Después de completar cada fase principal.
- Cuando el agente necesita tomar una decisión arquitectónica importante.
- Si el agente detecta ambigüedad en los requisitos.

Estrategias de corrección

1. Reinicio parcial del contexto

Si se detecta deriva, resumir el estado actual (qué se ha hecho, qué falta) y reiniciar la conversación con ese resumen como contexto inicial. Esto elimina el ruido acumulado.

2. Re-anclaje al objetivo original

Reformular explícitamente el objetivo original y las decisiones clave tomadas. Esto "recalibra" al agente.

3. División de la tarea

Si la tarea es demasiado larga para una sola sesión, dividirla en sub-tareas independientes, cada una con su propia sesión. Persistir el estado entre sesiones mediante commits y documentación.

Reanudación del trabajo sin repetir pasos ni diferir de decisiones anteriores

El patrón de reanudación robusta

Cuando un agente necesita reanudar un trabajo interrumpido, debe seguir este flujo:

1. LEER EL ESTADO PERSISTIDO
 - └ Revisar commits recientes en la rama de trabajo
 - └ Leer la descripción del PR (si existe)
 - └ Consultar comentarios del issue asociado
 - └ Verificar el estado del CI/CD
2. RECONSTRUIR EL CONTEXTO
 - └ Identificar qué pasos se completaron
 - └ Identificar qué decisiones se tomaron y por qué
 - └ Identificar qué falta por hacer
 - └ Verificar que el código actual es consistente con las decisiones
3. CONTINUAR DESDE EL ÚLTIMO PUNTO VÁLIDO
 - └ NO repetir pasos ya completados
 - └ NO contradecir decisiones anteriores (a menos que haya nueva información)
 - └ Completar los pasos pendientes en el orden establecido
 - └ Hacer commit del progreso incremental

Ejemplo práctico: Agente de codificación reanudando trabajo

Situación: El agente estaba implementando un sistema de autenticación y la sesión se interrumpió después de completar 3 de 5 pasos.

Estado persistido (en commits):

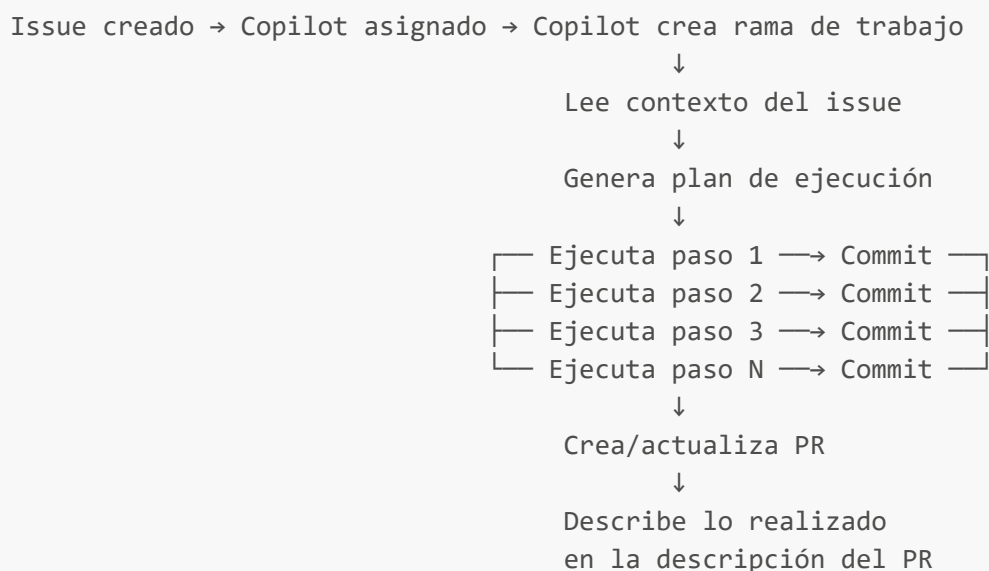
1. ☒ Commit: "feat: add User model with password hashing"
2. ☒ Commit: "feat: implement JWT token generation"
3. ☒ Commit: "feat: add login endpoint"
4. ☐ Pendiente: Agregar endpoint de registro
5. ☐ Pendiente: Agregar middleware de autenticación

Reanudación correcta: El agente lee los commits, identifica que los pasos 1-3 están completos, verifica que el código es consistente y continúa desde el paso 4.

Reanudación incorrecta (sin persistencia): El agente no tiene contexto previo, vuelve a implementar el modelo de usuario (duplicación), usa una librería diferente para JWT (inconsistencia) y genera conflictos con el código existente.

Aplicación práctica en GitHub

El flujo de persistencia del agente de codificación de Copilot



Cada commit es un punto de persistencia. Si el agente se interrumpe, puede leer los commits existentes y continuar.

Puntos clave para el examen

- ☒ **La deriva de contexto** es cuando el agente pierde alineación con la tarea original durante ejecuciones largas.
- ☒ **Causas principales de la deriva:** desbordamiento de la ventana de contexto, acumulación de información irrelevante, cambios en el entorno y sesgo de recencia.
- ☒ **Mecanismos de persistencia en GitHub:** git commits, descripciones de PR, comentarios en issues.
- ☒ **Los commits son el mecanismo principal** de persistencia del estado del agente de codificación.

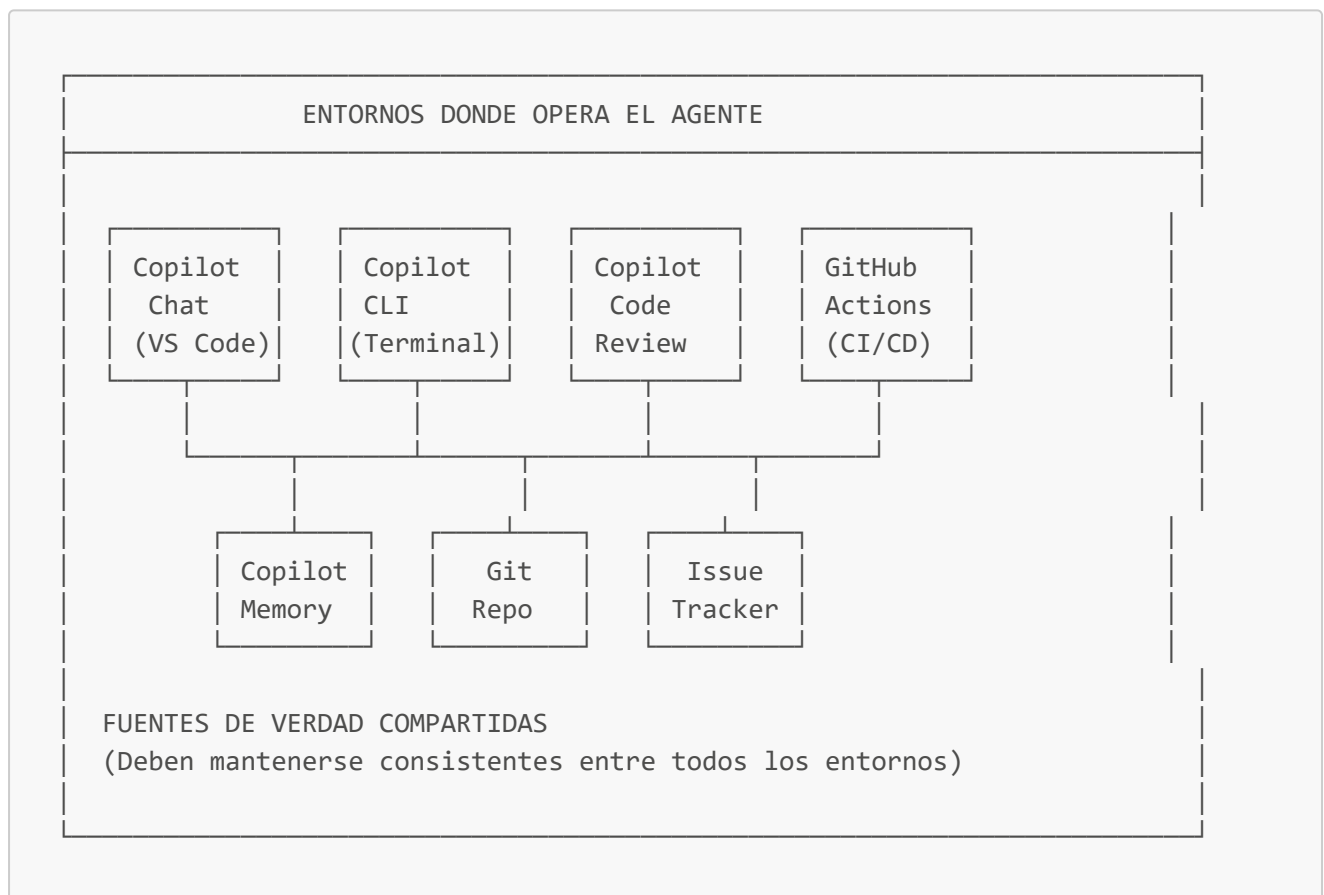
- ☒ **Reanudación robusta:** leer estado persistido → reconstruir contexto → continuar desde el último punto válido, sin repetir ni contradecir.
- ☒ **Detección de deriva:** anclas de objetivo, checkpoints de validación, revisión humana periódica.
- ☒ **Corrección de deriva:** reinicio parcial del contexto, re-anclaje al objetivo, división de la tarea en sub-tareas.
- ☒ **El principio fundamental:** el agente nunca debe repetir pasos ya completados ni tomar decisiones que contradigan las anteriores sin justificación explícita.

3.3 Garantizar la continuidad de la memoria y el estado del agente entre herramientas y entornos

Conceptos clave

El problema de la continuidad entre herramientas

Un agente moderno interactúa con **múltiples herramientas y entornos**. Cada uno puede tener su propia representación del estado, lo que crea desafíos de consistencia:



El desafío es que cada herramienta puede:

- **Leer estado desactualizado** si otra herramienta lo modificó.
- **Escribir estado conflictivo** si dos herramientas modifican lo mismo simultáneamente.
- **Perder contexto** al pasar de una herramienta a otra.

Compartir el estado del agente

Fuentes de verdad compartidas

Para mantener la continuidad, es necesario definir **fuentes de verdad únicas** (single sources of truth) que todas las herramientas compartan:

Fuente de verdad	Qué contiene	Acceso
Repositorio Git	Código fuente, configuración, instrucciones personalizadas	Lectura/escritura por todas las herramientas
Copilot Memory	Convenciones, preferencias, hechos del proyecto	Lectura por Copilot Chat, Code Review, CLI, coding agent
Issues y PRs	Requisitos, progreso, decisiones	Lectura/escritura por todos los entornos
GitHub Actions	Estado del CI/CD, resultados de tests	Lectura por el agente, escritura por el pipeline

Mecanismos de sincronización

1. Git como protocolo de sincronización

Git es el mecanismo principal para compartir estado entre herramientas:

- El agente en VS Code hace push de sus commits.
- El agente de codificación en la nube lee esos commits y continúa.
- GitHub Actions ejecuta CI/CD sobre el mismo estado.
- Code Review analiza los diffs del mismo repositorio.

2. Copilot Memory como contexto compartido

Copilot Memory actúa como un almacén de contexto compartido entre las diferentes interfaces de Copilot:

- Una convención almacenada por Copilot Chat en VS Code está disponible para Copilot CLI.
- Una preferencia de usuario es consistente entre VS Code, la web y la CLI.
- Las memorias de repositorio son visibles para todos los colaboradores con Copilot Memory habilitado.

3. APIs y webhooks para notificación de cambios

Cuando una herramienta modifica el estado, puede notificar a otras mediante:

- **Webhooks de GitHub:** Notifican eventos como push, PR creado, issue actualizado.
- **GitHub Actions:** Se activan automáticamente en respuesta a eventos.
- **Check runs y status checks:** Comunican el estado del CI/CD al agente.

Evitar contexto en conflicto

El **contexto en conflicto** ocurre cuando dos o más fuentes proporcionan información contradictoria al agente.

Causas comunes de conflicto

Causa	Ejemplo	Consecuencia
Instrucciones contradictorias	<code>copilot-instructions.md</code> dice "usar tabs" pero una memoria de Copilot dice "usar espacios"	El agente puede alternar entre ambos estilos
Estado desincronizado	El PR dice "paso 3 completo" pero el código no refleja el paso 3	El agente puede saltarse un paso o duplicarlo
Múltiples fuentes de verdad	La documentación dice una cosa, el código hace otra	El agente no sabe cuál seguir
Memorias obsoletas	Una memoria dice "el proyecto usa React 17" pero se actualizó a React 18	El agente genera código para la versión incorrecta

Estrategias para evitar conflictos

1. Jerarquía de precedencia explícita

Definir una jerarquía clara de qué fuente tiene prioridad cuando hay conflicto:

```
MÁXIMA PRIORIDAD
↓ Instrucciones explícitas del usuario (prompt actual)
↓ Archivos .instructions.md del directorio actual
↓ .github/copilot-instructions.md (nivel repositorio)
↓ Copilot Memory (hechos validados contra el código actual)
↓ Inferencia del código fuente
MÍNIMA PRIORIDAD
```

2. Principio de fuente única de verdad

Para cada tipo de información, designar **una sola fuente autoritativa**:

- **Convenciones de código** → `copilot-instructions.md` o linter config.
- **Arquitectura del proyecto** → Documentación en el repo (ADRs).
- **Estado del progreso** → Git commits y PR description.
- **Requisitos** → Issues de GitHub.

3. Validación cruzada

Cuando el agente detecta información potencialmente conflictiva, debe:

- Verificar contra el código fuente actual (la fuente de verdad definitiva).
- Preferir la información más reciente y verificable.

- Señalar el conflicto al usuario si no puede resolverlo automáticamente.

Evitar el contexto obsoleto

El **contexto obsoleto** (stale context) es información que fue correcta en el pasado pero ya no lo es. Es particularmente peligroso porque el agente no tiene forma inherente de saber que la información ha cambiado.

Fuentes de contexto obsoleto

Fuente	Cómo se vuelve obsoleta	Mecanismo de mitigación
Copilot Memory	El código cambia pero la memoria no se actualiza	Validación automática contra la rama actual; auto-eliminación a los 28 días
Archivos de instrucciones	El proyecto evoluciona pero las instrucciones no se actualizan	Revisión manual periódica; incluir en el proceso de code review
Comentarios en issues	Se toman decisiones nuevas que invalidan las anteriores	Actualizar el issue con las decisiones vigentes; cerrar issues obsoletos
Documentación del repo	La implementación cambia pero los docs no se actualizan	Tratar la documentación como código: actualizarla en los mismos PRs
Caché de sesión	El entorno cambia durante una sesión larga	Refrescar periódicamente la información del entorno

Estrategias para prevenir el contexto obsoleto

1. Validación en tiempo de uso (Validate-on-Use)

No confiar en información previamente almacenada sin verificarla:

- Copilot Memory implementa esto automáticamente al validar hechos contra la rama actual.
- El agente debería releer archivos relevantes antes de hacer cambios, no confiar en una lectura anterior de la sesión.

2. Marcas temporales y versionado

Asociar cada pieza de contexto con una marca temporal o versión:

- Los commits de git tienen timestamps y SHAs.
- Los comentarios de issues tienen fechas.
- Cuando el agente lee un archivo, debe considerar cuándo fue la última vez que se modificó.

3. Actualización proactiva

Mantener las fuentes de verdad actualizadas como parte del flujo de trabajo:

- Al hacer un PR que cambia la arquitectura, actualizar `copilot-instructions.md` en el mismo PR.
- Al cerrar un issue con una decisión diferente a la original, documentar el cambio.

- Incluir la actualización de documentación e instrucciones en las definiciones de "done".

4. Diseño para caducidad (Design for Staleness)

Asumir que toda información puede estar desactualizada y diseñar el sistema para manejarlo:

- Preferir instrucciones basadas en principios (que cambian raramente) sobre instrucciones basadas en detalles (que cambian frecuentemente).
- Ejemplo: "Seguir el patrón de manejo de errores del proyecto" es más resistente a la obsolescencia que "Usar try-catch con `AppError` en la línea 42 de `errors.ts`".

Aplicación práctica en GitHub

Escenario: Continuidad entre Copilot Chat, agente de codificación y CI/CD

PASO 1: Desarrollador usa Copilot Chat en VS Code

- Discute la solución para el issue #42
- Copilot almacena en Memory: "El issue #42 se resuelve añadiendo validación de email en el registro"
- El desarrollador asigna el issue al agente de codificación

PASO 2: Agente de codificación toma el issue

- Lee el issue #42 y sus comentarios
- Lee Copilot Memory: encuentra la memoria sobre validación de email
- Lee copilot-instructions.md: obtiene convenciones del proyecto
- Implementa la solución, hace commits, crea PR

PASO 3: GitHub Actions ejecuta CI/CD

- Los tests pasan (el agente siguió las convenciones)
- Los check runs reportan estado al PR

PASO 4: Copilot Code Review analiza el PR

- Lee las mismas instrucciones del repositorio
- Lee Copilot Memory para contexto adicional
- Genera revisión consistente con las convenciones del proyecto

RESULTADO: Todas las herramientas comparten el mismo contexto, evitando conflictos y obsolescencia.

Anti-patrón: Contexto fragmentado

✗ INCORRECTO:

- Convenciones de código en un wiki externo (inaccesible para el agente)
- Decisiones de diseño en mensajes de Slack (no persistidos)
- Estado del progreso solo en la mente del desarrollador (no compartido)
- Instrucciones diferentes en cada herramienta (inconsistencia)

☑ CORRECTO:

- Convenciones en `.github/copilot-instructions.md` (accesible por todas las herramientas)
- Decisiones de diseño en ADRs dentro del repositorio
- Estado del progreso en commits, PRs e issues de GitHub
- Una fuente de verdad única por cada tipo de información

Puntos clave para el examen

- ☒ **Compartir estado:** Git es el protocolo principal de sincronización entre herramientas. Copilot Memory proporciona contexto compartido entre interfaces de Copilot.
- ☒ **Copilot Memory es compartido** entre Copilot Chat, CLI, Code Review y el agente de codificación.
- ☒ **Contexto en conflicto:** ocurre cuando múltiples fuentes dan información contradictoria. Se resuelve con jerarquía de precedencia y fuente única de verdad.
- ☒ **Contexto obsoleto:** información que fue correcta pero ya no lo es. Se mitiga con validación en tiempo de uso (como hace Copilot Memory contra la rama actual).
- ☒ **Fuente única de verdad:** cada tipo de información debe tener una sola fuente autoritativa.
- ☒ **Validación contra la rama actual:** Copilot Memory verifica automáticamente que los hechos sigan siendo válidos antes de usarlos.
- ☒ **Anti-patrón:** almacenar contexto en lugares inaccesibles para el agente (wikis externos, mensajes de chat, conocimiento tácito).
- ☒ **Mejor práctica:** toda la información relevante debe estar en el repositorio (código, instrucciones, documentación, issues) donde el agente puede acceder a ella.
- ☒ **Diseñar para la caducidad:** preferir instrucciones basadas en principios sobre instrucciones basadas en detalles específicos.

Recursos

Documentación oficial de GitHub

- [About Copilot Memory](#) — Gestión de la memoria de Copilot.
- [Managing Copilot Memory for your organization](#) — Administración a nivel de organización.
- [Customizing Copilot with repository instructions](#) — Archivo `copilot-instructions.md`.
- [Using Copilot coding agent](#) — El agente de codificación y su gestión de estado.
- [About custom instructions for Copilot](#) — Archivos `.instructions.md`.

Conceptos clave a recordar

Concepto	Definición breve
Memoria a corto plazo	Contexto de la sesión actual; volátil
Memoria a largo plazo	Copilot Memory + archivos de instrucciones; persiste entre sesiones
Memoria externa	Archivos, BD, APIs que el agente consulta activamente
Deriva de contexto	Pérdida gradual de alineación con la tarea original

Concepto	Definición breve
Contexto en conflicto	Información contradictoria de múltiples fuentes
Contexto obsoleto	Información que fue correcta pero ya no lo es
Fuente única de verdad	Una sola fuente autoritativa por tipo de información
Validación en tiempo de uso	Verificar que la información sigue siendo válida antes de usarla
Auto-eliminación (28 días)	Copilot Memory elimina hechos no utilizados en 28 días
Persistencia via commits	Los git commits son el mecanismo principal de persistencia del agente